

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I M S GOWRISH SHANKAR REDDY(192424270) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for(i = 0; i <= n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1,2,3,4,5}
    int total;

    total = sumArray(arr);

    if(total == 15) {
        printf("Correct Sum\n")
    }

    int a = 10, b = 0;
    int c = a / b;

    int *p;
    *p = 20;

    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

ANS:

Line	Error
4	Missing semicolon
8	Illegal identifier #value
9	Missing semicolon
12	Wrong number of arguments
14	Assignment instead of comparison
15	Missing semicolon
20	Division by zero
23	Array index out of bounds
26	Invalid pointer dereference
29	Redundant +0
31	Infinite loop (x never changes)

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

ANS:

Error	Phase
#value	Lexical
Missing semicolons	Syntax
Wrong function arguments	Semantic
Assignment in condition	Intermediate Code / Logic
+0	Optimization
Division by zero	Runtime
Array overflow	Runtime
Pointer error	Runtime
Infinite loop	Runtime / Logical

(c) Compilation Behavior (Evaluation Level)

Compile Time

- Illegal identifier
- Missing semicolons
- Wrong function call

Execution Time

- Divide by zero
- Pointer crash
- Array overflow
- Infinite loop

Why Compilation Stops

Compiler cannot build the syntax tree if syntax is incorrect.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

ANS:

```
#include <stdio.h>
```

```
int divide(int a,int b)
{
    return a/b;
}
```

```
int main()
{
    int value=20;
    float num=15.5;
    int result;

    result=divide(10,2);

    if(result==5)
    {
        printf("Result is 5\n");
    }
}
```

```
int x=10;
int y=2;
```

```

int z=x/y;

int arr[4]={1,2,3,4};

printf("%d\n",arr[3]);

int temp=25;
int *ptr=&temp;

*ptr=25;

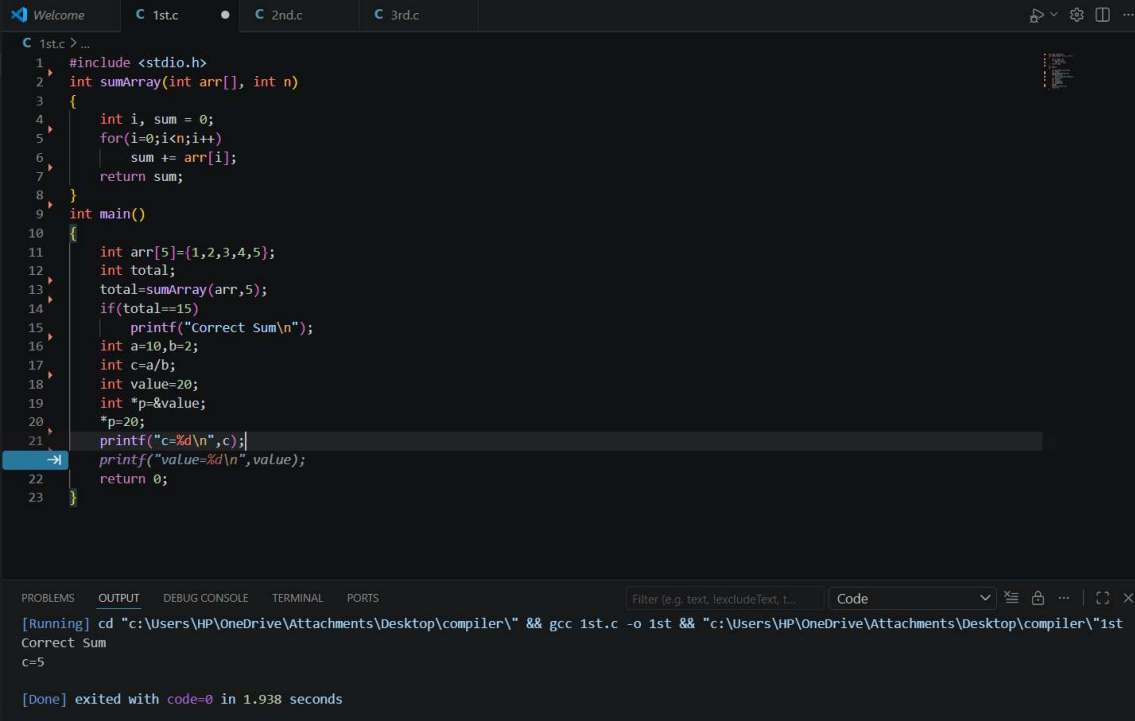
int total;

total=x+y*result;

while(x<20)
{
    printf("%d\n",x);
    x++;
}

return 0;
}

```



The screenshot shows a C program in a code editor with a terminal window at the bottom. The code defines a function `sumArray` to calculate the sum of an array, and a `main` function that uses it. The terminal shows the output of the program, which is "Correct Sum" and "c=5".

```

1  #include <stdio.h>
2  int sumArray(int arr[], int n)
3  {
4      int i, sum = 0;
5      for(i=0; i<n; i++)
6          sum += arr[i];
7      return sum;
8  }
9  int main()
10 {
11     int arr[5]={1,2,3,4,5};
12     int total;
13     total=sumArray(arr,5);
14     if(total==15)
15         printf("Correct Sum\n");
16     int a=10,b=2;
17     int c=a/b;
18     int value=20;
19     int *p=&value;
20     *p=20;
21     printf("c=%d\n",c);
22     printf("value=%d\n",value);
23     return 0;
}

```

Terminal Output:

```

[Running] cd "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\" && gcc 1st.c -o 1st && "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\1st
Correct Sum
c=5

[Done] exited with code=0 in 1.938 seconds

```

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.

2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

ANS:

Pointer Error

Uninitialized pointer accesses random memory.

Effects:

- Crash
- Data corruption
- Security vulnerabilities

Array Bounds

`arr[4]`

Valid indices:

0 1 2 3

Accessing index 4 reads memory beyond the array, leading to undefined behavior.

Operator/Intermediate Code

`if(result = 5)`

Generates assignment rather than comparison.

Result becomes 5, making the condition always true.

Correct:

`if(result == 5)`

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

ANS:

Detect Runtime Errors Earlier

1. Static analysis to detect:
 - Division by zero
 - Null/uninitialized pointers
 - Array bounds violations
2. Symbolic execution to evaluate possible execution paths and flag unsafe operations before runtime.

Improve Debugging

- Display exact line and column numbers.
- Suggest likely fixes (e.g., "Did you mean == instead of =?").
- Group related errors to reduce cascading diagnostics.

Optimization Improvements

The optimizer can eliminate redundant expressions such as:

```
total = x + y * result + 0;
```

Optimized form:

```
total = x + y * result;
```

It can also perform constant folding, dead-code elimination, and remove unnecessary computations to generate more efficient target code.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
```

```
int main() {  
    int lnum = 100;  
    float rate@ = 25.5;  
    char ch = 'abc';  
    int total = 50$20;  
    int num#1 = 30;  
    int %count = 40;  
  
    printf("Welcome to Compiler Design);  
  
    int x = 09;  
    int y = 0xZ12;  
    int z = 12.3.4;  
  
    char str1[] = "Hello;  
    char str2[] = "World";  
  
    int _valid = 10;  
    int invalid-id = 20;  
  
    float value = 5..6;  
    int data = 10@5;  
  
    char c = 'xy';  
    int 3value = 60;  
  
    printf("Value: %d\n", total);  
  
    return 0;  
}
```

Tasks

(a) Error Identification

- i) Identify all lexical errors in the program.
- ii) List the line number and the invalid token.

ANS:

Line No.	Invalid Token	Error Description
4	1num	Identifier starts with a digit
5	rate@	Identifier contains illegal character @
6	'abc'	Character constant contains more than one character
7	\$	Illegal symbol/operator
8	num#1	Identifier contains illegal character #
9	%count	Identifier starts with illegal symbol %
11	"Welcome to Compiler Design	Missing closing double quotation mark
13	09	Invalid octal constant (digit 9 not allowed in octal)
14	0xZ12	Invalid hexadecimal constant (Z is invalid)
15	12.3.4	Malformed floating-point constant
17	"Hello	Missing closing double quotation mark
21	invalid-id	Hyphen (-) is not allowed in identifiers
23	5..6	Malformed floating-point constant
24	@	Illegal symbol/operator
26	'xy'	Character constant contains multiple characters
27	3value	Identifier starts with a digit

Line	Invalid Token
4	1num
5	rate@
6	'abc'
7	\$
8	#
9	%count
11	Unterminated string literal
13	09
14	0xZ12
15	12.3.4
17	Unterminated string literal
21	invalid-id
23	5..6
24	@
26	'xy'
27	3value

(b) Error Classification

- For each error, explain why it is considered a lexical error.
- Specify which rule of token formation is violated:

- iii) Identifier rules
- iv) Constant formation
- v) String/character literal rules
- vi) Operator/symbol rules

ANS:

A lexical error occurs when the source code contains **invalid characters or tokens** that violate the rules of the C language before parsing begins.

Invalid Token	Reason
1num	Identifiers cannot begin with digits.
rate@	@ is not permitted in identifiers.
'abc'	Character literals must contain only one character.
50\$20	\$ is not a valid C operator.
num#1	# is not allowed inside identifiers.
%count	Identifiers cannot begin with %.
Missing "	String literal is not terminated.
09	9 is invalid in an octal constant.
0xZ12	Z is not a hexadecimal digit.
12.3.4	Floating-point constant has two decimal points.
invalid-id	Hyphen is treated as the subtraction operator, not part of an identifier.
5..6	Invalid floating-point constant.
10@5	@ is an illegal symbol.
'xy'	Character literal contains multiple characters.
3value	Identifiers cannot begin with digits.

(ii) Rules Violated

Error	Rule Violated
1num, 3value	Identifier Rules
rate@, num#1, %count, invalid-id	Identifier Rules
09, 0xZ12, 12.3.4, 5..6	Constant Formation Rules
'abc', 'xy', missing "	String/Character Literal Rules
\$. @	Operator/Symbol Rules

(c) Token Correction

- i) Rewrite each invalid token into a valid token.
- ii) Example: 2value → value2

ANS:

Invalid Token	Correct Token
1num	num1
rate@	rate
'abc'	'a'
50\$20	50 + 20

num#1	num1
%count	count
"Welcome to Compiler Design"	"Welcome to Compiler Design"
09	9
0xZ12	0x12
12.3.4	12.34
"Hello	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10 + 5
'xy'	'x'
3value	value3

(d) Compiler Behavior Analysis

- i) Explain how a lexical analyzer (scanner) processes such errors.
- ii) Discuss whether the compiler:
- iii) Stops immediately
- iv) Continues scanning using error recovery

The screenshot shows a C program in a code editor with the following code:

```

1  #include <stdio.h>
2  int main() {
3      int num1 = 100;
4      float rate = 25.5;
5      char ch = 'a';
6      int total = 50 + 20;
7      int num1_value = 30;
8      int count = 40;
9      printf("Welcome to Compiler Design\n");
10     int x = 9;
11     int y = 0x12;
12     float z = 12.34;
13     char str1[] = "Hello";
14     char str2[] = "World";
15     int _valid = 10;
16     int invalid_id = 20;
17     float value = 5.6;
18     int data = 10 + 5;
19     char c = 'x';
20     int value3 = 60;
21     printf("Value: %d\n", total);
22     return 0;
23 }

```

The terminal output shows the program running successfully:

```

[Running] cd "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\" && gcc 3rd.c -o 3rd && "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\3rd
Welcome to compiler Design
Value: 70
[Done] exited with code=0 in 0.604 seconds

```

(i) How does a lexical analyzer process these errors?

The lexical analyzer (scanner) reads the source code character by character and groups them into valid tokens such as:

- Keywords
- Identifiers
- Constants
- Operators
- Delimiters

If an invalid token is encountered, the scanner reports a **lexical error**.

Flow of lexical analysis:

Source Program



Lexical Analyzer (Scanner)



Valid Tokens



Syntax Analyzer (Parser)

If an invalid token is found (for example, 1num), the scanner reports an error and attempts to continue scanning to find additional errors.

(ii) Does the compiler stop immediately?

Not always.

Modern compilers generally use **error recovery** techniques.

Error Recovery

- Reports the lexical error.
- Skips or replaces the invalid token.
- Continues scanning the remaining program.
- Displays multiple errors in one compilation instead of stopping after the first error.

Example:

Error: Invalid identifier '1num' at line 4.

Continuing lexical analysis...

This helps programmers correct several errors at once.

(e) Advanced Thinking

- i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
- ii) Suggest how error messages can be improved for beginners.

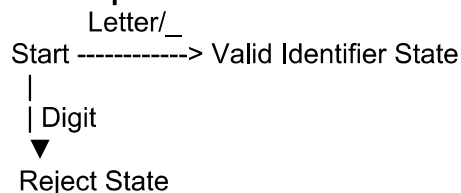
ANS:

(i) DFA idea to detect identifiers starting with digits

A valid C identifier must:

- Begin with a letter (A–Z, a–z) or underscore (_)
- Followed by letters, digits, or underscores

DFA Representation



If the first character is a digit, the DFA immediately enters the **Reject State**, indicating an invalid identifier.

(ii) Improved Error Messages for Beginners

Instead of displaying generic messages like:

Error: Invalid token.

A compiler can provide more informative messages, such as:

Old Message

Improved Message

Invalid identifier	Identifier cannot start with a digit. Try num1 instead of 1num.
Illegal character	@ is not allowed in identifiers. Remove or replace it.
Malformed number	Floating-point constant contains more than one decimal point.
Unterminated string	Missing closing double quotation mark (").
Invalid character constant	Character literals must contain exactly one character.

These descriptive messages help beginners understand the mistake and fix it more quickly.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

- A. Lexical Errors
- B. Syntax Errors
- C. Semantic Errors
- D. Intermediate Code Generation Issues (logic/precedence)
- E. Code Optimization Issues (redundancy/inefficiency)
- F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
    return a / b
}

int main() {
    int #value = 20;
    float num = 15.5
    int result;

    result = divide(10);

    if(result = 5) {
        printf("Result is 5\n")
    }

    int x = 10;
    int y = 0;
    int z = x / y;

    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[4]);

    int *ptr;
    *ptr = 25;
```

```

int total;
total = x + y * result + 0;

while(x < 20) {
    printf("%d\n", x);
}

return 0;
}

```

Do the following Tasks

(a) Identify Errors (Analysis Level)

- I. Identify **all errors** in the program.
- II. Mention **line number + error description**.

Line No.	Error	Description
4	Missing semicolon	; missing after return a / b
8	#value	Invalid identifier containing #
9	Missing semicolon	; missing after float num = 15.5
12	Wrong function call	divide() requires two arguments but only one is supplied
14	Assignment instead of comparison	Uses = instead of ==
15	Missing semicolon	; missing after printf()
20	Division by zero	y = 0, causing runtime error
23	Array index out of bounds	arr[4] accesses memory beyond the array
26	Uninitialized pointer	Dereferencing ptr without initialization
29	Redundant expression	+ 0 has no effect
31–33	Infinite loop	x is never incremented inside the loop

(ii) Summary of Errors

- **Lexical Error: 1**
- **Syntax Errors: 3**
- **Semantic Error: 1**
- **Intermediate Code Issue: 1**
- **Optimization Issue: 1**
- **Runtime Errors: 4**

(b) Classify by Compiler Phase

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

ANS:

Error	Compiler Phase
#value	Lexical Error
Missing semicolons	Syntax Error
Wrong function arguments	Semantic Error
if(result = 5)	Intermediate Code / Logical Issue
+0 in expression	Optimization Issue
Division by zero	Runtime Error
Array out of bounds	Runtime Error
Invalid pointer access	Runtime Error
Infinite loop	Runtime / Logical Error

(c) Compilation vs Execution (Evaluation)

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.

ANS:

(i) Errors Detected at Compile Time

- Invalid identifier (#value)
- Missing semicolons
- Incorrect number of function arguments
- Assignment used in conditional (compiler warning)

(ii) Errors Detected During Execution

- Division by zero
- Array index out of bounds
- Invalid pointer dereference
- Infinite loop

(iii) Why Some Errors Prevent Compilation

Compilation proceeds through these phases:

Lexical Analysis



Syntax Analysis



Semantic Analysis



Code Generation



Execution

If a lexical or syntax error occurs, the compiler cannot correctly build the program structure (parse tree). Therefore, later phases such as semantic analysis and code generation may not be performed until those errors are corrected.

(d) Correct the Program (Application)

- I. Rewrite the **fully corrected program**.
- II. Ensure:
 - A. Proper syntax
 - B. Correct semantics
 - C. Safe execution

ANS:

```
#include <stdio.h>
```

```
int divide(int a, int b)
{
    return a / b;
}
```

```
int main()
{
    int value = 20;
    float num = 15.5;
    int result;

    result = divide(10, 2);

    if(result == 5)
    {
        printf("Result is 5\n");
    }
}
```

```
int x = 10;
int y = 2;
```

```
int z = x / y;

int arr[4] = {1,2,3,4};
printf("%d\n", arr[3]);

int temp = 25;
int *ptr = &temp;
*ptr = 25;

int total;
total = x + y * result;

while(x < 20)
{
    printf("%d\n", x);
    x++;
}

printf("Division Result = %d\n", z);
printf("Total = %d\n", total);

return 0;
}
```

```
C 2nd.c > main()
1  #include <stdio.h>
2  int divide(int a,int b){
3      return a/b;}
4  int main(){
5      int value=20;
6      float num=15.5;
7      int result;
8      result=divide(10,2);
9      if(result==5)
10         printf("Result is 5\n");
11     int x=10;
12     int y=2;
13     int z=x/y;
14     int arr[4]={1,2,3,4};
15     printf("%d\n",arr[3]);
16     int data=25;
17     int *ptr=&data;
18     *ptr=25;
19     int total=x+y*result;
20     while(x<20)
21     {
22         printf("%d\n",x);
23         x++;
24     }
25     return 0;
26 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t...

```
[Running] cd "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\" && gcc 3rd.c -o 3rd && "c:\Users\HP\OneDr
Welcome to Compiler Design
Value: 70

[Done] exited with code=0 in 0.604 seconds
```

(e) Deep Analysis (Evaluation)

- I. Explain how **incorrect pointer usage** affects runtime.
- II. Analyze how **array bounds violation** impacts memory.
- III. Discuss how **operator precedence errors** affect intermediate code.

ANS:

(i) Effect of Incorrect Pointer Usage

Example:

```
int *ptr;
*ptr = 25;
```

Impact

- ptr does not point to valid memory.
- Dereferencing it causes **undefined behavior**.
- The program may:
 - Crash (Segmentation Fault)
 - Corrupt memory
 - Produce unpredictable results

Correct approach:

```
int value = 25;
```

```
int *ptr = &value;
```

(ii) Impact of Array Bounds Violation

Example:

```
int arr[4] = {1,2,3,4};
```

```
printf("%d", arr[4]);
```

Valid indices are:

```
arr[0]
```

```
arr[1]
```

```
arr[2]
```

```
arr[3]
```

arr[4] accesses memory outside the array.

Effects

- Undefined behavior
- Incorrect output
- Reading unrelated memory
- Possible program crash

(iii) Operator Precedence / Intermediate Code Issues

Example:

```
if(result = 5)
```

Here, = performs **assignment**, not comparison.

Intermediate code generated assigns 5 to result, making the condition always evaluate as true.

Correct statement:

```
if(result == 5)
```

This generates proper comparison instructions in intermediate code.

(f) Advanced Task (Create Level)

- I. Suggest **2 improvements** in compiler design to:
 - A. Detect runtime errors earlier
 - B. Improve debugging messages
- II. Propose how **optimization phase** can remove redundant expressions.

(i) Improvements in Compiler Design

A. Detect Runtime Errors Earlier

1. **Static Analysis**
 - Detect division by zero.
 - Detect use of uninitialized pointers.
 - Warn about array index violations.
2. **Symbolic Execution**
 - Analyze possible execution paths.
 - Predict runtime failures before execution.

B. Improve Debugging Messages

Instead of:

Error near line 20

Display:

Line 20: Division by zero detected.

Variable 'y' has value 0.

Consider checking the divisor before division.

Another example:

Line 12: Function 'divide()' expects 2 arguments but only 1 was provided.

These messages help programmers locate and fix errors quickly.

(ii) Optimization Phase Improvements

The compiler's optimization phase can improve efficiency by removing redundant computations.

Example

Original:

```
total = x + y * result + 0;
```

Optimized:

```
total = x + y * result;
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts

Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand
-----------------------------------	-----	--	-----------------------------------	-----------------------------------	---